

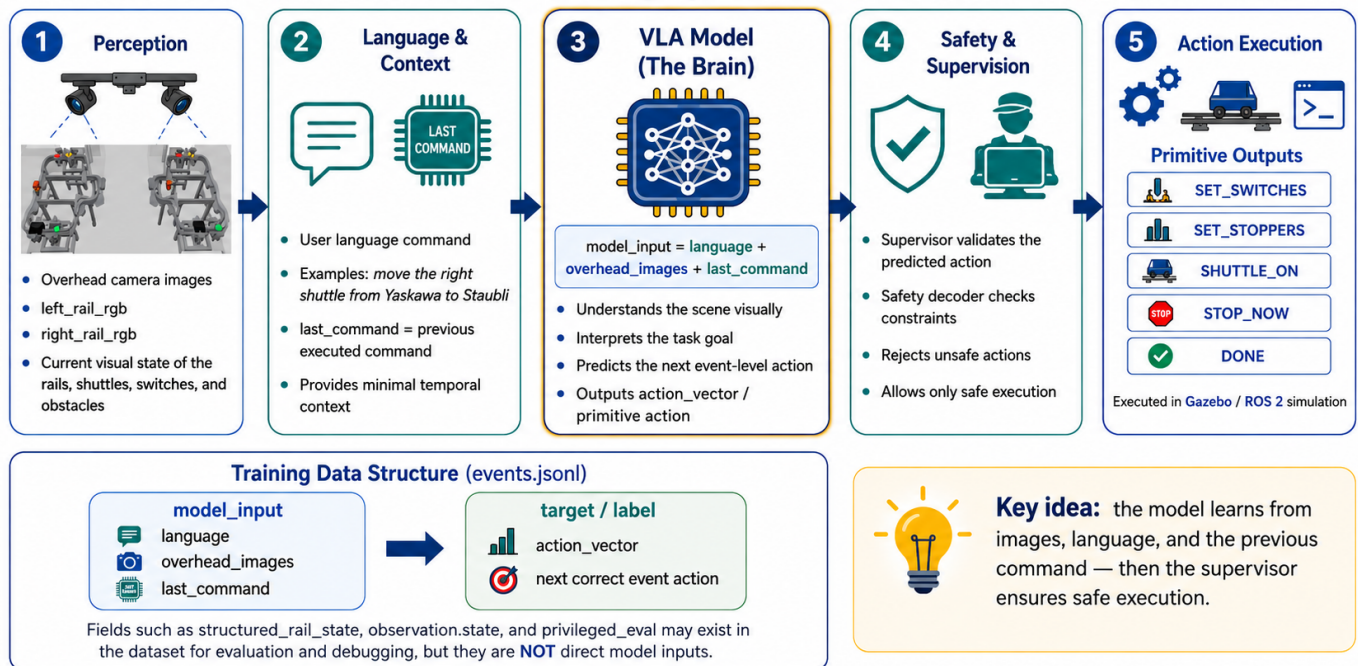
Room 315 VLA - Training Design and Meeting Summary

System Architecture Overview

The diagram summarizes the full Room 315 VLA pipeline: perception, language context, VLA model prediction, safety supervision, and safe execution in Gazebo / ROS 2.

Room 315 VLA System Architecture

From Vision and Language to Safe Action Execution



Stage	Explanation
1. Perception	Receives overhead camera images from the left and right rails: left_rail_rgb and right_rail_rgb. These images describe the current visual state of the rails, shuttles, switches, and obstacles.
2. Language & Context	The language command defines the task goal, while last_command provides minimal temporal context by indicating the previously executed action.
3. VLA Model	The model receives language, overhead_images, and last_command, then predicts the next event-level action as an action_vector or equivalent primitive action.
4. Safety & Supervision	The supervisor validates the predicted action and the safety decoder rejects unsafe commands before execution.
5. Action Execution	Validated actions are executed in Gazebo / ROS 2. Typical primitives include SET_SWITCHES, SET_STOPPERS, SHUTTLE_ON, STOP_NOW, and DONE.

Key idea: the model learns from images, language, and the previous command. The supervisor is responsible for safe execution. Fields such as structured_rail_state, observation.state, and privileged_eval may exist for evaluation and debugging, but they are not direct model inputs.

Meeting Summary

1. The design of the Room 315 VLA system was presented, including the mechanism for converting images, the language command, and the last executed command into an action_vector.

2. The `model_input` was defined to include only: `language`, `overhead_images`, and `last_command`.
3. It was clarified that the `action_vector` represents the training target and should not be part of the `model_input` in order to avoid answer leakage.
4. The importance of `last_command` was explained in distinguishing visually similar states, such as the difference between selecting `STOP_NOW` and selecting `DONE`.
5. The issue of directly providing the system state or the shuttle position to the model was discussed.
6. The idea of using simulation to generate images with true labels, such as the shuttle position, was adopted so that the model can learn to estimate the position from the image.
7. It was clarified that simulation information can be used as labels or for evaluation, but not as direct input to the model.
8. The issue of multiple shuttles was discussed, with an initial focus on visually detecting the presence of more than one shuttle, while leaving individual shuttle identity as a later step.
9. The use of PlanSys/PDDL was introduced to automatically generate scenarios instead of relying on manually written ones.
10. The next practical direction was defined as generating scenarios through PlanSys/PDDL, executing them in ROS/Gazebo, and then recording `events.jsonl` to train the VLA model.

1. General Idea

Each line in `events.jsonl` becomes one training sample. The sample is divided into two main parts: input X and target Y.

Input X	VLA Model	Target Y
<code>language</code> <code>right_image</code> <code>left_image</code> <code>last_command</code>	Processes language, images, and previous command. Predicts the next event-level action.	<code>action_vector</code> next correct event action

Core idea: `model_input` is the question, and `action_vector` is the correct answer. The model sees the question, predicts an answer, and the training process compares the prediction with the correct answer.

2. How the File Is Used for Training

The training code reads one row and builds `x` from `model_input`. It builds `y` from `action_vector`.

```
x = {
  "language": row["model_input"]["language"],
  "right_image": dataset_root / row["model_input"]["overhead_images"]["right_rail_rgb"],
  "left_image": dataset_root / row["model_input"]["overhead_images"]["left_rail_rgb"],
  "last_command": row["model_input"]["last_command"]
}

y = row["action_vector"]
```

Field	Meaning	Purpose
<code>language</code>	The user command, for example: move the right shuttle from Yaskawa to Staubli.	It tells the model the task goal.
<code>right_image</code>	The right overhead camera image.	It shows the visual state of the right rail.
<code>left_image</code>	The left overhead camera image.	It shows the visual state of the left rail.
<code>last_command</code>	The previous command executed before the current decision.	It provides minimal temporal context.

Data leakage warning: the correct action must exist in the dataset, but it must stay outside `model_input`. If it is placed inside `model_input`, the model can copy the answer instead of learning from the images, language, and previous command.

3. The Role of model_input

model_input defines the clean interface for the learned model. It contains only the information the deployed model is allowed to use.

```
"model_input": {
  "language": "move right shuttle from yaskawa to staubli",
  "last_command": {
    "action": "START"
  },
  "overhead_images": {
    "left_rail_rgb": "episodes/.../left_rail_rgb/000004.jpg",
    "right_rail_rgb": "episodes/.../right_rail_rgb/000004.jpg"
  }
}
```

This means the model learns from the user goal, the camera images, and the previous command. It does not directly receive privileged simulator state.

4. The Role of action_vector

action_vector is the structured target. It represents the correct next event-level action. During training, the model predicts this vector, and the loss compares the prediction to the true vector.

```
primitive_id      = y[0]
side_id          = y[1]
switch_masks     = y[2:6]
switch_values    = y[6:10]
stopper_masks    = y[10:14]
stopper_values   = y[14:18]
speed_mps        = y[18]
wait_condition_id = y[19]
target_id        = y[20]
reason_id        = y[21]
```

Part	Meaning	Example
primitive_id	The main type of action.	STOP_NOW, SHUTTLE_ON, DONE
side_id	The rail side.	right or left
switch_masks	Which switches should be changed.	A3 mask = 1 means change A3
switch_values	The desired switch states.	exterior or interior
stopper_masks	Which stoppers should be changed.	A1/A2/A3/A4 masks
stopper_values	The desired stopper states.	open or closed
speed_mps	The shuttle speed in meters per second.	0.3 m/s
wait_condition_id	What the supervisor should wait for after executing the command.	wait until shuttle command is applied
target_id	The target object or component.	right_shuttle, switches, terminal
reason_id	Why this action was selected.	shuttle_start, shuttle_stop, task_succeeded

5. Why Split y Into Heads?

The action vector contains different kinds of values. Some are categorical, some are binary masks, and speed can be numeric. For this reason, the model output can be split into several prediction heads.

```

primitive_id    = y[0]      # classification head
side_id        = y[1]      # classification head
switch_masks   = y[2:6]    # binary heads
switch_values  = y[6:10]   # categorical heads
stopper_masks  = y[10:14]  # binary heads
stopper_values = y[14:18]  # categorical heads
speed_mps      = y[18]     # regression or discrete class
wait_condition_id = y[19]  # classification head
target_id      = y[20]     # classification head
reason_id      = y[21]     # classification head

```

Benefit: this makes training clearer, debugging easier, and error analysis more precise. For example, it becomes possible to identify whether the model made a mistake in the primitive, the side, the speed, or the target.

6. Masks vs Values

Masks and values are separated to prevent accidental state changes.

Concept	Question it answers	Encoding
Mask	Should this component be changed?	0 = keep unchanged, 1 = apply the value
Value	What new state should this component take?	switch: exterior/interior; stopper: open/closed

```

# Change only switch A3 to INTERIOR
switch_masks = [0, 0, 1, 0]
switch_values = [0, 0, 2, 0]

```

Here, only A3 is changed. A1, A2, and A4 remain unchanged because their masks are zero.

7. Auxiliary and Debug Fields

Some fields may exist in the dataset for training support, evaluation, or debugging. They are not direct model inputs.

Field	Role	Direct model input?
auxiliary_targets	Optional helper labels for auxiliary tasks, such as visual state prediction.	No. They are helper labels, not main inputs.
structured_rail_state	Readable state for audits and debugging.	No.
privileged_eval	Privileged evaluation labels or simulator state.	No.
observation.state	State vector for evaluation, debugging, or baselines.	No, not for the deployable VLA policy.

Correct usage: use `model_input` as the input and `action_vector` as the target. Use `auxiliary_targets` only as optional helper labels. Use `structured_rail_state` and `privileged_eval` only for evaluation, explanation, and debugging.

8. Simple Training Loop

```

for row in events_jsonl:
    x = build_model_input(row)
    y = row["action_vector"]

    prediction = model(x)
    loss = compare(prediction, y)
    update_model_weights(loss)

```

The model learns the mapping:

```
language + overhead_images + last_command -> next correct action_vector
```

9. Proposed Training Scenarios

This section lists recommended scenario families for training the Room 315 VLA model. The objective is to cover diverse event-level decisions instead of one fixed route.

Code	Scenario name	Why it is useful for training
r01	move_right_shuttle_full_exterior_loop	Teaches a complete exterior-loop movement on the right rail.
r02	move_right_shuttle_from_yaskawa_to_staubli	Teaches right-side station-to-station transport from Yaskawa to Staubli.
r03	move_right_shuttle_from_staubli_to_yaskawa	Teaches the reverse right-side route from Staubli back to Yaskawa.
r06-r09	Stop the right shuttle at stoppers A1..A4.	Teaches precise stopping at different stopper locations.
r10-r13	Right rail interior-loop entry, movement, and transitions.	Teaches switching between exterior-loop and interior-loop behavior on the right rail.
l01-l13	Equivalent scenario families for the left rail.	Teaches the same behavior types on the left side and improves left/right generalization.
m02	emergency_stop_all	Teaches global emergency/safety stopping behavior.
m08	left_slot3_kuka_then_slot2	Teaches sequencing between shuttle motion and robot motion.
m10	right_obstacle_aware_route	Teaches visual obstacle-aware movement on the right rail.
m11	left_obstacle_aware_route	Teaches visual obstacle-aware movement on the left rail.

Current m Scenarios

Code	What it does	Recording notes
m02	Starts an emergency stop all episode, prepares the right rail, runs the shuttle briefly, sends stop_all, and waits for the stop to settle.	Useful for a clear safety command: stop all motion.
m08	Runs left_slot3_kuka_then_slot2: move the left shuttle to slot 3, send the KUKA joint pose for 4 seconds, then move the left shuttle to slot 2.	Tests rail + robot sequencing, not only shuttle motion.
m10	Runs right_obstacle_aware_route. The shuttle makes a large exterior-loop lap. If the right visual obstacle blocks the path, the shuttle stops before it; otherwise it completes the loop.	Stop distance is controlled by right_manual_obstacle_stop_before_m, default 0.20 m.
m11	Same as m10, but on the left rail with the left visual obstacle.	The stop point is computed from the obstacle pose.

Training coverage: relying on a single route is not sufficient for this type of VLA training. A balanced dataset should include right-rail, left-rail, stopper, loop-transition, emergency, robot-sequencing, and obstacle-aware scenarios. This diversity supports general event-level decision making and reduces the risk of memorizing one fixed sequence.

Dataset principle: each scenario should still follow the same clean training structure: model_input contains only the language command, overhead images, and the previous command, while action_vector remains the target label.